

Neural Thickets

A Plain-English Guide to the Paper

Part 1 — Paper Summary

The Big Idea in One Sentence

After a large language model has been pretrained, you can improve it on specific tasks simply by **randomly tweaking its weights** — no gradient descent, no backpropagation required — and then combining the best random tweaks together.

Background: Why This Seems Crazy

When you train a neural network, you're searching through an astronomically large space of possible weight configurations. A modern LLM might have 7 billion numbers (parameters) that all need to be set just right. The standard wisdom — going back to a 2001 paper the authors quote — is that random guessing in such a space is completely hopeless.

Gradient descent is the standard solution: compute exactly which direction to nudge each weight to reduce error, and take millions of small steps in that direction. It's structured, intelligent search.

The Key Observation: The "Thicket"

The authors noticed something surprising about **large, well-pretrained models**. Imagine the model's performance on a task as a landscape, where height = accuracy.

- **Small/untrained models** sit on a flat plain, with high-accuracy "peaks" far away. You need gradient descent to navigate there. This is the **needle-in-a-haystack** regime.
- **Large pretrained models** sit in a valley surrounded by many nearby peaks. Good solutions are dense all around you. This is the **thicket** regime.

Analogy: Imagine finding a coffee shop. In the Arizona desert (small model), you need a map and a car — random wandering won't work. But in downtown Manhattan (large pretrained model), you can walk in almost any direction and stumble upon a great one.

What They Actually Measure

Solution density: if you add a small random noise vector to all model weights, what's the probability the result is *better* than the original on a given task?

- **0.5B parameter model:** essentially 0% of random perturbations improve math performance.

- **32B parameter model:** about **64%** of random perturbations improve performance.

Nearly two-thirds of random weight jitters make the big model better at math. That's a staggering shift.

The Diversity Discovery: Specialists, Not Generalists

Each perturbation tends to be a **specialist**. A perturbation that makes the model better at math tends to make it *worse* at chemistry. One that improves creative writing tends to hurt programming. They're not finding an all-around better model — they're finding a neighborhood full of **diverse experts**.

Analogy: Imagine a consulting firm. Your star generalist employee (the pretrained model) is good at everything but master of none. Hidden in the building are hundreds of specialists — one who's amazing at accounting but terrible at marketing, one who's a brilliant writer but can't code. RandOpt finds those specialists and has them vote on problems together.

The Algorithm: RandOpt

Given this landscape, the authors build a simple two-step algorithm:

- **Step 1 — Random Guessing (Training):** Sample $N=5,000$ random perturbations of the model weights. Evaluate each on a small training set. Keep only the top $K=50$.
- **Step 2 — Ensembling (Inference):** When given a question, run it through all 50 top models. Take a majority vote on the answer.

That's it. No gradients. No optimizer. No backward passes. All N models can be evaluated simultaneously on separate GPUs — entirely parallel.

Concrete result: On the Countdown math task with OLMo-3-7B-Instruct, RandOpt raised accuracy from ~65% to 70% in just **3.2 minutes** on a 200-GPU cluster.

How Does It Compare to Real Training Methods?

Surprisingly well. Across seven benchmarks (math, coding, writing, chemistry) and six models (0.5B–8B parameters), RandOpt with $K=50$ **matches or beats** PPO, GRPO, and Evolution Strategies under equal compute budgets.

The catch: at inference time, RandOpt requires 50 forward passes instead of 1. The authors address this with **distillation** — training a single new model to imitate all 50 specialists, recovering most performance at normal cost.

What's Actually Being Fixed?

The authors honestly decompose improvements on GSM8K math into:

- **Reasoning thicket (~12%):** The model was getting math wrong before, now gets it right.
- **Format thicket (~19%):** The model knew the answer but was formatting it wrong (e.g., not using the "#####" tag), and now formats correctly.
- **Regression (~0.7%):** Cases where the adapted model got worse.

Notably, this same split (real reasoning + formatting) is also true of GRPO and other methods.

The Deeper Implication

We usually think of a pretrained model as a single thing — one point in weight space. But the authors argue you should think of it as **a distribution over many specialized models**, all clustered nearby. Pretraining doesn't just find one good solution; it navigates to a neighborhood *rich with solutions*.

This reframes post-training (fine-tuning, RLHF, etc.) as less about "teaching the model something new" and more about **selecting and amplifying capabilities already latent** in the pretrained neighborhood.

Key Limitations

- Works only with large, well-pretrained models (~1.5B+ parameters minimum).
- Requires K=50 forward passes at inference time unless you distill.
- Performance saturates — you can't keep adding perturbations forever.
- Majority voting doesn't work well for open-ended generation (stories, images), though preliminary extensions to diffusion models are shown.

Part 2 — Background: What Is a Pretrained Model?

The Basic Idea

A neural network starts as essentially **random noise**. Every weight is initialized to a small random number. At this point, it can't do anything useful — ask it a question and it outputs gibberish.

Training is the process of adjusting all those weights, using gradient descent, until the model gets good at some task. This can take weeks on thousands of GPUs and millions of dollars of compute.

A **pretrained model** is simply one that has already gone through this expensive training process. Someone else did the hard work; you get the result as a starting point.

How Pretraining Actually Works for LLMs

For large language models specifically, pretraining usually means one task: **predict the next word**.

The model reads hundreds of billions of sentences from the internet, books, code, Wikipedia, and more, and at each step tries to predict what word comes next. Every time it's wrong, the weights get nudged slightly in a better direction. After trillions of such nudges, something remarkable happens — to predict text well, the model is forced to learn:

- Grammar and syntax
- Facts about the world
- How to reason logically
- How code works, how chemistry is discussed, how stories are structured
- ...and vastly more, all as a *side effect* of next-word prediction.

Analogy: Imagine teaching someone to be a great conversation partner by having them read every book, article, forum post, and textbook ever written. They never explicitly studied 'how to do math' — but after reading millions of math textbooks, they absorbed it anyway.

Why "Pre" Training?

The "pre" means this training happens *before* you use the model for your specific purpose. It's a general-purpose foundation. After pretraining, you typically either **fine-tune** it on a smaller specific dataset (cheaper, since the model already "knows things"), or use it directly with a well-crafted prompt.

The Weights Are the Model

A model like Llama-3-8B has **8 billion weights** — just 8 billion floating point numbers stored in a file. That's it. When you "run" the model, you load those numbers into memory and do matrix multiplications with them.

Pretraining is the process that determined what those 8 billion numbers should be. Once found, the file can be shared with anyone — which is what open-source models like Llama, OLMo, and Qwen are.

Why This Matters for the Paper

Think of the 8 billion weights as a **single point in an 8-billion-dimensional space**. Pretraining navigated to a particular point in that space. The paper asks: *what does the neighborhood around that point look like?*

For small or untrained models, the neighborhood is barren. For large pretrained models, it's a **thicket** — packed with useful variants in every direction. And the reason is precisely because pretraining on diverse data forced the weights into a location simultaneously near many different specialized solutions.

This is why pretraining is the prerequisite. If you try RandOpt on a model that hasn't been pretrained, it completely fails — the thicket only exists because pretraining put you in the right neighborhood first.

Summary prepared from the paper: *Neural Thickets: Diverse Task Experts Are Dense Around Pretrained Weights* — Yulu Gan & Phillip Isola, MIT CSAIL (arXiv:2603.12228)